Not All Neighbors are Friendly: Learning to Choose Hop Features to Improve Node Classification

Sunil Kumar Maurya
Tokyo Institute of Technology
AIRC, AIST
Tokyo, Japan
skmaurya@net.c.titech.ac.jp

Xin Liu
AIRC, AIST
Tokyo, Japan
xin.liu@aist.go.jp

Tsuyoshi Murata
Tokyo Institute of Technology
AIRC, AIST
Tokyo, Japan
murata@c.titech.ac.jp

ABSTRACT

The fundamental operation of Graph Neural Networks (GNNs) is the feature aggregation step performed over neighbors of the node based on the structure of the graph. In addition to its own features, the node gets additional combined features from its neighbors for each hop. These aggregated features help define the similarity or dissimilarity of the nodes with respect to the labels and are useful for tasks like node classification. However, in real-world data, features of neighbors at different hops may not correlate with the node's features. Thus, any indiscriminate feature aggregation by GNN might cause the addition of noisy features leading to degradation in model's performance. In this work, we show that selective aggregation leads to better performance than default aggregation on the node classification task. Furthermore, we propose Dual-Net GNN architecture with a classifier model and a selector model. The classifier model trains over a subset of input node features to predict node labels while the selector model learns to provide optimal input subset to the classifier for best performance. These two models are trained jointly to learn the best subset of features that give higher accuracy in node label predictions. With extensive experiments, we show that our proposed model outperforms the state-of-the-art GNN models with remarkable improvements up to 27.8%. Source code available at https://github.com/sunilkmaurya/DualNetGNN

CCS CONCEPTS

• **Computing methodologies** → **Neural networks**.

KEYWORDS

Graph Neural Networks, Node Classification

ACM Reference Format:
Sunil Kumar Maurya, Xin Liu, and Tsuyoshi Murata. 2022. Not All Neighbors are Friendly: Learning to Choose Hop Features to Improve Node Classification. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM '22), October 17–21, 2022, Atlanta, GA, USA.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3511808.3557543

CIKM '22, October 17–21, 2022, Atlanta, GA, USA

ACM ISBN 978-1-4503-9236-5/22/10…$15.00
https://doi.org/10.1145/3511808.3557543

1 INTRODUCTION

Recent years have seen rapid growth of Graph Neural Networks (GNNs) being utilized on real-world applications in multitude of domains. These applications range in wide variety of tasks like recommendation systems [7, 41, 47], graph mining [24, 48], molecular properties predictions[10, 17, 27] , natural language processing[12, 46], vision tasks[39, 45], node ranking[14, 28, 29] etc.

GNNs are designed to operate on graph structure derived from the dataset and learn from node/edge features. The primary component of GNNs which separates them from other neural network architectures is the feature aggregation operation. The neighbors of the nodes are explored via the graph structure, and their features are combined with the node's features. The underlying assumption is that nodes are connected to other nodes with similar properties in the graph (homophily). Hence combining features from the neighbors helps to improve the signal, which helps in the learning process. Recently, there have been some works that have shown that in some datasets, dissimilar nodes are connected in the graph (heterophily) [31, 51, 52] and they may require different aggregation schemes for the design of the GNN.

The aggregation is performed in K steps, where K is the number of hops from the source node to the neighbors in the graph. Thus, the extent of neighborhood feature aggregation expands with K. In this way, in addition to its own features, the node gets a set of features based on its neighborhood. There are many different ways of aggregation schemes proposed in current GNN literature. Many GNN models use simple indiscriminate aggregation [18, 40, 43], some use random sampling of neighbors [3, 13], attention-based aggregation [36, 49], filter coefficient based aggregation [4–6] etc.

Real-world data often have noisy components and do not follow the assumptions of GNN models strictly. For example, all neighbors of the node may not be similar to it in feature information especially considering nodes lying at various hop-distance away from the source node. Thus, any feature aggregation may often cause the addition of noisy features and may reduce the signal present in node features to learn effectively. Researchers have explored many ways to tackle this challenge by proposing many improvements in model design, for example, making models more deeper [4, 11, 20], using attention layers [36], residual layers [4, 19, 44], etc. However, most of these solutions attempt to fit the model to the noisy data in an effective way [42, 50].

Instead of designing a GNN model to fit the noisy data, we propose to treat the problem with feature selection approach. We identify that not all features generated via aggregation steps are useful, and often using these less informative features can be detrimental to the performance of the GNN model. We experimentally

demonstrate that learning certain subsets of these features can significantly improve the performance on a wide variety of datasets. Based on these observations, we propose a Dual-Net GNN model which learns to select node features generated by aggregation at various hops. The model consists of two neural networks: *classifier* and *selector*. The role of the classifier is to predict the labels of the nodes based on a given input. The role of the selector is to predict the subset of input features on which the classifier will have the best performance. These two networks are trained jointly, providing feedback to each other. Following are the main contributions of our work:

- We are the first to run extensive experiments on standard node classification datasets to show that feature selection can improve the GNN model's performance.
- We propose a novel dual-net GNN architecture to find the optimal subset of features which leads to the better prediction accuracy of the model.
- In experimental results, our proposed model outperforms the state-of-the-art (SOTA) GNN models on the node classification task and achieves up to 27.8% higher accuracy.

2 PRELIMINARIES

Let $G = (V, E)$ be an undirected graph with n nodes and m edges. The graph is represented as adjacency matrix denoted by $A \in \{0, 1\}^{n \times n}$ with each element $A_{ij} = 1$ if there exists an edge between node v_i and v_j, otherwise $A_{ij} = 0$. When self-loops are added to the graph, then the resultant adjacency matrix is denoted as $\tilde{A} = A + I$. Most GNNs use a self-looped adjacency matrix, however recent publications have shown that a simple/no-loop adjacency matrix is useful for learning heterophily properties in graph [26, 52]. Each node is associated with a d-dimensional feature vector, and the feature matrix for all nodes is represented as $X \in \mathbb{R}^{n \times d}$. The aggregation step is performed with matrix multiplication of adjacency matrix and feature matrix. For K-step aggregation, aggregated features are calculated as $A^k X, k \in (1 \cdots K)$. We denote $\mathcal{X} = \{X, X_1, X_2, X_3 ..., X_K\}$ as a set of node feature matrices generated after aggregation step including initial node feature matrix.

3 FEATURE IMPORTANCE IN LEARNING

3.1 Importance of Feature Subsets

In this section, we conduct experiments on node classification datasets to verify the impact of combining node features on the GNN model's performance.

For our experiments, we use a two-layered MLP model. As we analyze both heterophily and homophily properties in a graph, we calculate both self-looped and no-loop features of the nodes. Hence for $K = 3$, our feature set has total of 7 (1+2×3) different features for the node: $\mathcal{X} = \{X, AX, (A + I)X, A^2X, (A + I)^2X, A^3X, (A + I)^3X\}$. We are interested in finding how different combinations of these input features affect node classification performance on various datasets. We train our model in three settings.

(1) Single setting: Input is single feature matrix from $\mathcal{X}$.
(2) All setting: All matrices in $\mathcal{X}$ are used as input.
(3) Subset setting: Model is trained on all combinations of features (PowerSet($\mathcal{X}$)$\backslash\emptyset$) and the best result is reported. One example of such a subset is $\{X, (A + I)X, A^3X\}$.

Table 1 shows the results of our experiments. We observe that homophily datasets like Cora, Citeseer, and Pubmed benefit from feature aggregation with self-looped adjacency matrices, while datasets like Chameleon and Squirrel benefit from no-loop adjacency matrices (Single setting). For datasets like Wisconsin, Texas, and Cornell, the node's own features are most informative for GNN performance. However, when we aggregate all features from $\mathcal{X}$, we observe improvements in homophily datasets, while other datasets show a significant reduction in accuracy (All setting). It implies that for most of these datasets, adding all features results in the addition of noisy and non-informative features, which leads to the failure of the GNN model to learn effectively. Following the Subset setting, we observe that using a subset of features achieves the best performance in all datasets.

Our experimental results show that with a certain optimal subset of node feature matrices, GNNs can achieve higher performance on the node classification task. However, calculating model's performance for all possible feature subsets is computationally expensive, especially with higher hops due to combinatorial explosion and practically not feasible.

3.2 Feature Selection in GNN

Similar problem has been explored in machine learning literature using various feature selection strategies [23, 38]. However, with GNNs, it is more challenging in two ways. First, node features in many datasets are sparse. For example, most datasets in Table 2 have more than 95% sparsity for node feature matrix. Second, the dimensionality of node features increases with each aggregation step due to multiple feature matrices after aggregation. Hence it becomes challenging to use feature selection algorithms, and there is a need for new approaches to tackle this problem.

3.3 Problem Formulation

Given the input $\mathcal{X}$ as a set of $2K + 1$ node feature matrices and $\mathcal{Y}$ as labels of the nodes. We are interested in identifying a subset of $\mathcal{X}$ of length up to $p < 2K + 1$, for which the GNN model gives the best performance. All such possible subsets are defined as $\mathcal{M} = \binom{2K+1}{p}$. Let $m \in \mathcal{M}$ denote any one of such possible subsets. Our objective is to identify the optimal subset m^* among all subsets.

4 DUAL-NET GNN ARCHITECTURE

4.1 Model Description

We propose a dual network GNN architecture for optimal feature selection. The first network is *classifier* a two-layered MLP, similar to the one used in Section 3.1 for predicting labels of the nodes. The second network is *selector*, also a two-layered MLP, yet with a different input/output configuration. The role of the selector is to find out an optimal feature subset of $\mathcal{X}$ based on the performance feedback of the classifier. Both networks are trained jointly in an alternate manner.

The classifier network is parameterized with θ, represented as $f_c(\theta; \mathcal{X}, m)$. The input to the network is a subset of node feature matrices, $m \in \mathcal{M}$. In the first layer, each input matrix is linearly transformed and the output is summed. The non-linearity (ReLU) is applied and then mapped to the second layer. After training of the joint model, $f_c(\theta^*; \mathcal{X}, m^*)$ is applied on the test split of the dataset,

Table 1: Classification Accuracy on node classification task on 2-layered MLP with hidden dimension as 64. Results with Subset setting (blue) are compared with best results (underlined) in other input feature settings.

	Single Feature							All	Subset
Dataset	**X**	**AX**	**(A + I)X**	**A^2X**	**$(A+I)^2$X**	**A^3X**	**$(A+I)^3$X**		
Cora	73.40	79.55	84.28	83.86	85.47	83.58	85.41	87.5	**88.04**
Citeseer	71.66	69.10	73.53	72.38	74.07	70.55	73.92	77.09	**77.43**
Pubmed	87.79	81.77	88.27	84.70	88.06	83.06	86.63	89.55	**89.83**
Chameleon	46.05	77.74	71.22	76.07	71.77	75.26	71.62	72.25	**78.55**
Wisconsin	87.45	63.13	58.03	62.54	52.94	60.00	51.76	79.8	**88.62**
Texas	85.40	66.21	61.35	67.29	58.64	62.43	58.10	78.91	**88.91**
Cornell	85.94	58.64	63.51	58.64	61.62	58.91	60.27	72.25	**86.75**
Squirrel	30.24	73.18	63.79	71.28	63.37	64.42	62.82	64.68	**73.12**
Actor	35.32	25.47	29.22	25.38	27.95	25.27	26.43	35.39	**35.67**

where θ^* denotes learned parameters of the classifier and m^* is the optimal input subset of node feature matrices.

The selector model is parameterized with ϕ, represented as $f_s(\phi, m)$. The input to the selector network is a one-hot encoding vector $\overrightarrow{m}$ of dimension $2K+1$ representing the subset of feature matrices to be selected as per m. For example, for $K = 3$, the subset $\{X, (A+I)X, A^3X\}$ has $\overrightarrow{m} = (1, 0, 1, 0, 0, 1, 0)$. The output is a single scalar value. The purpose of the selector net is to learn to predict the average performance of the classifier to the mask corresponding to the input subset.

4.2 Loss functions

The classifier is trained with simple Cross-Entropy loss commonly used in the node classification task. For each training step, the input is $m \in \mathcal{M}$. In the implementation, it represents the masking operation of input features to the model.

$$\mathcal{L}_c(X, m; \theta) = \frac{1}{|V|} l(X, m, y; \theta) \tag{1}$$

The loss of selector is MSE (mean squared error) loss between the output of selector and the classifier loss. The selector learns to map the input masks to the classifier's loss. Thus the output of the selector varies with different input subset masks in congruence with the classifier's performance. With a good input subset, the classifier will have a lower loss, and correspondingly selector will have a lower output value and vice versa.

$$\mathcal{L}_s(\overrightarrow{m}; \phi) = \left(f_s(\overrightarrow{m}; \phi) - \mathcal{L}_c(X, m; \theta)\right)^2 \tag{2}$$

4.3 Training Methodology

The training of Dual-Net is multi-step process as presented below:
Step 1: First step is the exploration stage, where we train the classifier and selector on combinations of node feature matrices. For each forward pass, a random subset m sampled from $\mathcal{M}$, and corresponding node feature matrices are used as input to the classifier. Similarly, the corresponding mask vector is set as input to the selector. Cross-entropy loss for the classifier is calculated and backpropagated. For the selector, MSE loss is calculated between the output of selector O_s and loss of classifier $\mathcal{L}_c$ and backpropagated. The training is continued in this manner till the classifier yields different losses for different mask combinations stably and selector learns to map these masks to classifier's performance. The number of training epoches is set as a hyperparameter.
Step 2: In this step, our objective is to exploit the learned selector to identify indices of useful feature matrices and generate possible

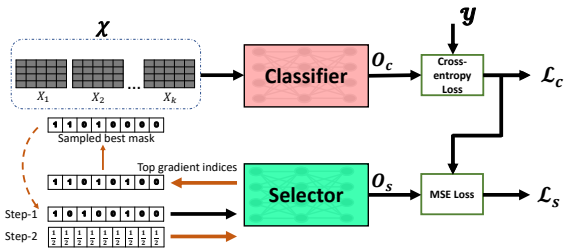

Figure 1: Figure shows model operation in Step 2, classifier and selector are trained jointly to identify optimal subset of feature matrices

Table 2: Statistics of the node classification datasets

Datasets	Hom. Ratio	Nodes	Edges	Features	Classes
Cora	0.81	2,708	5,429	1,433	7
Citeseer	0.74	3,327	4,732	3,703	6
Pubmed	0.80	19,717	44,338	500	3
Chameleon	0.23	2,277	36,101	2,325	4
Wisconsin	0.21	251	499	1,703	5
Texas	0.11	183	309	1,703	5
Cornell	0.30	183	295	1,703	5
Squirrel	0.22	5,201	198,353	2,089	5
Actor	0.22	7,600	26,659	932	5

optimal masks for the classifier. In machine learning literature, given a trained model, gradients with respect to the input are often used to identify important input components [15, 33, 35]. The input components with larger gradients contribute more toward model's output. Similarly, we aim to identify important indices of the input mask vector to the trained selector model. The node feature matrices corresponding to these indices would lead to better performance of the classifier model.

We use a mask vector with all indices initialized with equal initial weights $(\frac{1}{2} \cdots \frac{1}{2})$ as input to the selector. We then calculate gradients with respect to the input vector. Indices with top-p gradients are identified, however, the optimal subset m^* might be a smaller subset of these indices, i.e. $len(m^*) \leq p$. A fixed number of combinations of these indices are sampled, and validation loss on the classifier is calculated. Mask with the lowest loss is selected, and both classifier and selector are trained on it for one epoch. This step is repeated a fixed number of times, s.
Step 3: After Step 2, the input mask with the least validation loss is identified as m^*, and training is continued with only the classifier till the stopping criterion is satisfied.

Table 3: Mean classification accuracy (%) on fully-supervised node classification task. Results for GCN, GAT, GraphSAGE, Cheby+JK, MixHop and H2GCN-1 are taken from [52]. For GEOM-GCN, GCNII, TDGNN, WRGAT, and GloGNN++, results are taken from the respective article. Best performance for each dataset is marked as bold.

	Cora	Citeseer	Pubmed	Chameleon	Wisconsin	Texas	Cornell	Squirrel	Actor	Mean Acc.
GCN	87.28±1.26	76.68±1.64	87.38±0.66	59.82±2.58	59.80±6.99	59.46±5.25	57.03±4.67	36.89±1.34	30.26±0.79	61.62
GAT	82.68±1.80	75.46±1.72	84.68±0.44	54.69±1.95	55.29±8.71	58.38±4.45	58.92±3.32	30.62±2.11	26.28±1.73	58.55
GraphSAGE	86.90±1.04	76.04±1.30	88.45±0.50	58.73±1.68	81.18±5.56	82.43±6.14	75.95±5.01	41.61±0.74	34.23±0.99	69.50
Cheby+JK	85.49±1.27	74.98±1.18	89.07±0.30	63.79±2.27	82.55±4.57	78.38±6.37	74.59±7.87	45.03±1.73	35.14±1.37	69.89
MixHop	87.61±0.85	76.26±1.33	85.31±0.61	60.50±2.53	75.88±4.90	77.84±7.73	73.51±6.34	43.80±1.48	32.22±2.34	68.10
GEOM-GCN	85.27	**77.99**	90.05	60.90	64.12	67.57	60.81	38.14	31.63	64.05
GCNII	88.01±1.33	77.13±1.38	**90.30±0.37**	62.48±2.74	81.57±4.98	77.84±5.64	76.49±4.37	N/A	N/A	-
H2GCN-1	86.92±1.37	77.07±1.64	89.40±0.34	57.11±1.58	86.67±4.69	84.86±6.77	82.16±4.80	36.42±1.89	35.86±1.03	70.71
TDGNN-w	88.01±1.32	76.58±1.40	89.22±0.41	46.31±2.42	85.57±3.78	83.00±4.50	82.92±6.61	26.73±1.47	37.11±0.96	68.38
WRGAT	88.20±2.26	76.81±1.89	88.52±0.92	65.24±0.87	86.98±3.78	83.62±5.50	81.62±3.90	48.85±0.78	36.53±0.77	72.93
GPRGNN	**88.49±0.95**	77.08±1.63	88.99±0.40	66.47±2.47	85.88±3.70	86.49±4.83	81.89±6.17	49.03±1.28	36.04±0.96	73.37
GloGNN++	88.33±1.09	77.22±1.78	89.24±0.39	71.21±1.84	88.04±3.22	84.05±4.90	85.95±5.81	57.88±1.76	**37.70±1.40**	75.51
Dual-Net GNN	87.77±1.16	77.15±1.58	89.64±0.43	**78.46±0.66**	**89.41±3.64**	**87.57±3.24**	**88.11±5.69**	**73.97±1.89**	37.29±0.80	**78.81**
Training Time	(36.53 s)	(46.64 s)	(71.58 s)	(59.95 s)	(19.45 s)	(46.83 s)	(44.84 s)	(88.55 s)	(52.13 s)	

5 RELATED WORKS

The problem of node classification has been extensively studied in last few years. [3, 4, 6, 11, 13, 16, 18–20, 22, 25, 30, 31, 37, 44, 51] have proposed various architectures to improve the performance of GNNs. Most recent improvements have been to make GNN models deeper [4, 11, 20], robust for homophily and heterophily in datasets [1, 31, 51] and mitigate the over-smoothing phenomenon [4, 19, 32]. Most GNNs use self-looped adjacency matrix as input, [26, 52] has proposed that also using simple adjacency matrix is useful for heterophily datasets. In many different machine learning fields, the problem of discarding noisy features to improve performance have been approached using feature selection [2, 9, 21, 34, 38].

6 EXPERIMENTS

In this section, we discuss the results of experiments with our model on real-world node classification datasets.

6.1 Datasets

We perform experiments on nine datasets commonly used in GNN literature (Table 2). Homophily ratio [52] denotes the fraction of edges which connects two nodes of the same label. A higher value (closer to 1) indicates strong homophily, while a lower value (closer to 0) indicates strong heterophily in the dataset. To provide a fair comparison, we use publicly available data splits taken from [31]. We compare the experiment results of other GNN models with our proposed model on this same split.

6.2 Settings and Baselines

For a fully-supervised node classification task, each dataset is split evenly for each class into 48%, 32%, and 20% for training, validation, and testing [31, 52]. We report the performance as mean classification accuracy over ten random splits. We compare our model to 12 different baselines and use the published results as the best performance of these models. For models with multiple variants, we choose the one with the best performance on most datasets. GPRGNN uses random splits in their published results. For a fair comparison, we ran their publicly available code on our standard splits while keeping other settings the same. Hidden dimension of the model is set as 64, K as 3, p as 4, and s as 20.

6.3 Results and Discussion

6.3.1 Results. Table 3 shows the comparison of the mean classification accuracy of our model and other GNN models. Comparing the results with other GNNs, we observe that our model overcomes the effect of noisy features and significantly improves the performance on Chameleon (+10.2%), Wisconsin (+1.6%), Texas (+4.19%), Cornell (+2.51%) and Squirrel (+27.8%) datasets. For Cora (-0.8%), Citeseer (-1.1%), Pubmed (-0.7%), and Actor (-1.1%), the performance of our model is at par with other GNN models.

6.3.2 Training Time. Due to its simple architecture, the training time of our model is relatively low. On commonly used datasets Cora, Citeseer, and Pubmed, the average training time of our model (51.5 s) is comparable to GPRGNN (53.8 s), while GCNII (923.8 s) takes roughly 18 times more due to its deep architecture.

6.3.3 Scalability. We pre-compute feature matrices before training the model. This approach is similar to some GNN works [8, 40] which have shown scalability for large datasets. Similarly, our model can also be scaled for large datasets by implementing to take input features in batches.

6.3.4 Future Work. In this work, we highlighted the importance of feature selection and propose the model using two simple MLP networks. However, more sophisticated architectures can be used to further improve the performance. There is also a scope for using our approach to other graph-related applications. We consider it as a direction for our future work.

7 CONCLUSION

In this work, we demonstrate that feature aggregation steps in GNN can generate noisy features for nodes, which in turn can reduce the performance of the GNN model. We tackle this challenge by proposing a Dual-Net GNN architecture to learn the optimal subset of features for better performance of the classifier on the node classification task. With extensive experiments, we show that our proposed model can reduce the effect of noisy features and outperform the SOTA GNN models.

ACKNOWLEDGEMENT

This work was supported by JSPS Grant-in-Aid for Scientific Research (Grant No. 21K12042, 17H01785), and the NEDO (Grant No. JPNP20006).

REFERENCES

[1] Deyu Bo, Xiao Wang, Chuan Shi, and Huawei Shen. 2021. Beyond Low-frequency Information in Graph Convolutional Networks. In *AAAI*.

[2] Girish Chandrashekar and Ferat Sahin. 2014. A survey on feature selection methods. *Computers & Electrical Engineering* 40, 1 (Jan. 2014), 16–28.

[3] Jie Chen, Tengfei Ma, and Cao Xiao. 2018. FastGCN: Fast Learning with Graph Convolutional Networks via Importance Sampling. In *ICLR*.

[4] M. Chen, Zhewei Wei, Zengfeng Huang, B. Ding, and Y. Li. 2020. Simple and Deep Graph Convolutional Networks. *ICML* (2020).

[5] Eli Chien, Jianhao Peng, Pan Li, and O. Milenkovic. 2021. Adaptive Universal Generalized PageRank Graph Neural Network. *ICLR* (2021).

[6] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. *arXiv:1606.09375* (2016).

[7] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph Neural Networks for Social Recommendation. In *The World Wide Web Conference (WWW '19)*. Association for Computing Machinery, New York, NY, USA, 417–426.

[8] Fabrizio Frasca, Emanuele Rossi, Davide Eynard, Ben Chamberlain, Michael Bronstein, and Federico Monti. 2020. SIGN: Scalable Inception Graph Neural Networks. *arXiv:2004.11198* (2020).

[9] K. Fukumizu and Chenlei Leng. 2012. Gradient-based kernel method for feature extraction and variable selection. In *NIPS*.

[10] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70 (ICML'17)*.

[11] Jonathan Godwin, Michael Schaarschmidt, Alexander Gaunt, Alvaro Sanchez-Gonzalez, Yulia Rubanova, Petar Veličković, James Kirkpatrick, and Peter Battaglia. 2021. Very Deep Graph Neural Networks Via Noise Regularisation. *arXiv:2106.07971* (2021).

[12] Zhijiang Guo, Yan Zhang, and Wei Lu. 2019. Attention Guided Graph Convolutional Networks for Relation Extraction. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Florence, Italy, 241–251.

[13] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *NIPS*. 1024–1034.

[14] Yixuan He, Quan Gan, David Wipf, Gesine Reinert, Junchi Yan, and Mihai Cucuringu. 2022. GNNRank: Learning Global Rankings from Pairwise Comparisons via Directed Graph Neural Networks. In *ICML*.

[15] Yotam Hechtlinger. 2016. Interpretation of Prediction Models Using the Input Gradient. *arXiv:1611.07634* (Nov. 2016).

[16] Wei Jin, Tyler Derr, Yiqi Wang, Yao Ma, Zitao Liu, and Jiliang Tang. 2021. Node Similarity Preserving Graph Convolutional Networks *(WSDM '21)*. Association for Computing Machinery, 148–156.

[17] Wengong Jin, Kevin Yang, Regina Barzilay, and Tommi Jaakkola. 2019. Learning Multimodal Graph-to-Graph Translation for Molecular Optimization. *arXiv:1812.01070* (Jan. 2019).

[18] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. *ICLR* (2017).

[19] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Predict then Propagate: Combining neural networks with personalized pagerank for classification on graphs.

[20] Guohao Li, Matthias Müller, Bernard Ghanem, and V. Koltun. 2021. Training Graph Neural Networks with 1000 Layers. In *ICML*.

[21] Jundong Li, Kewei Cheng, Suhang Wang, Fred Morstatter, Robert P. Trevino, Jiliang Tang, and Huan Liu. 2017. Feature Selection: A Data Perspective. *Comput. Surveys* 50, 6 (Dec. 2017), 94:1–94:45.

[22] Xiang Li, Renyu Zhu, Yao Cheng, Caihua Shan, Siqiang Luo, Dongsheng Li, and Weining Qian. 2022. *Finding Global Homophily in Graph Neural Networks When Meeting Heterophily*. Technical Report arXiv:2205.07308. arXiv.

[23] Yifeng Li, Chih-Yu Chen, and Wyeth W. Wasserman. 2015. Deep Feature Selection: Theory and Application to Identify Enhancers and Promoters. In *Research in Computational Molecular Biology (Lecture Notes in Computer Science)*, Teresa M. Przytycka (Ed.). Springer International Publishing, Cham, 205–217.

[24] Yujia Li, Chenjie Gu, T. Dullien, Oriol Vinyals, and P. Kohli. 2019. Graph Matching Networks for Learning the Similarity of Graph Structured Objects. In *ICML*.

[25] Sitao Luan, Chenqing Hua, Qincheng Lu, Jiaqi Zhu, Mingde Zhao, Shuyuan Zhang, Xiao-Wen Chang, and Doina Precup. 2021. *Is Heterophily A Real Nightmare For Graph Neural Networks To Do Node Classification?* Technical Report arXiv:2109.05641. arXiv.

[26] Yao Ma, Xiaorui Liu, Neil Shah, and Jiliang Tang. 2021. Is Homophily a Necessity for Graph Neural Networks? *arXiv:2106.06134* (2021).

[27] Kaushalya Madhawa, Katushiko Ishiguro, Kosuke Nakago, and Motoki Abe. 2019. GraphNVP: An Invertible Flow Model for Generating Molecular Graphs. *arXiv:1905.11600* (2019).

[28] Sunil K. Maurya, Xin Liu, and Tsuyoshi Murata. 2019. Fast Approximations of betweenness centrality using Graph Neural Networks. In *International Conference on Information and Knowledge Management (CIKM '19)*.

[29] Sunil Kumar Maurya, Xin Liu, and Tsuyoshi Murata. 2021. Graph Neural Networks for Fast Node Ranking Approximation. *ACM Transactions on Knowledge Discovery from Data* 15, 5 (May 2021), 78:1–78:32.

[30] Sunil Kumar Maurya, Xin Liu, and Tsuyoshi Murata. 2022. Simplifying approach to node classification in Graph Neural Networks. *Journal of Computational Science* 62 (2022).

[31] Hongbin Pei, Bingzhe Wei, K. Chang, Yu Lei, and B. Yang. 2020. Geom-GCN: Geometric Graph Convolutional Networks. *ICLR* (2020).

[32] Y. Rong, W. Huang, Tingyang Xu, and Junzhou Huang. 2020. DropEdge: Towards Deep Graph Convolutional Networks on Node Classification. In *ICLR*.

[33] A. Ross, Michael C. Hughes, and Finale Doshi-Velez. 2017. Right for the Right Reasons: Training Differentiable Models by Constraining their Explanations. In *IJCAI*.

[34] Jiliang Tang, Salem Alelyani, and Huan Liu. 2014. Feature selection for classification: A review. *Data Classification: Algorithms and Applications* (Jan. 2014), 37–64.

[35] Michael Tsang, Dehua Cheng, and Yan Liu. 2018. Detecting Statistical Interactions from Neural Network Weights. *arXiv:1705.04977* (Feb. 2018).

[36] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. *ICLR* (2018).

[37] Petar Velickovic, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R. Devon Hjelm. 2019. Deep Graph Infomax. *ICLR*.

[38] Maksymilian Wojtas and Ke Chen. 2020. Feature Importance Ranking for Deep Learning. *arXiv:2010.08973* (Oct. 2020).

[39] Sanghyun Woo, Dahun Kim, Donghyeon Cho, and In So Kweon. 2018. LinkNet: relational embedding for scene graph. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems (NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 558–568.

[40] Felix Wu, Amauri H. Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Q. Weinberger. 2019. Simplifying Graph Convolutional Networks. In *ICML*.

[41] Shiwen Wu, Fei Sun, Wentao Zhang, and Bin Cui. 2021. Graph Neural Networks in Recommender Systems: A Survey. *arXiv:2011.02260* (April 2021).

[42] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2019. A Comprehensive Survey on Graph Neural Networks. *arXiv:1901.00596* (Dec. 2019).

[43] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks? *ICLR*.

[44] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation Learning on Graphs with Jumping Knowledge Networks. PMLR.

[45] Jianwei Yang, Jiasen Lu, Stefan Lee, Dhruv Batra, and Devi Parikh. 2018. Graph R-CNN for Scene Graph Generation. In *Computer Vision – ECCV 2018 (Lecture Notes in Computer Science)*, Vittorio Ferrari, Martial Hebert, Cristian Sminchisescu, and Yair Weiss (Eds.). Springer International Publishing, Cham, 690–706.

[46] Yongjing Yin, Fandong Meng, Jinsong Su, Chulun Zhou, Zhengyuan Yang, Jie Zhou, and Jiebo Luo. 2020. A Novel Graph-based Multi-modal Fusion Encoder for Neural Machine Translation. In *ACL*.

[47] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *KDD '18*.

[48] Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L. Hamilton, and Jure Leskovec. 2018. Hierarchical Graph Representation Learning with Differentiable Pooling *(NIPS'18)*. 4805–4815.

[49] Jiani Zhang, Xingjian Shi, Junyuan Xie, Hao Ma, Irwin King, and D. Yeung. 2018. GaAN: Gated Attention Networks for Learning on Large and Spatiotemporal Graphs. In *UAI*.

[50] Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2019. Graph Neural Networks: A Review of Methods and Applications. *arXiv:1812.08434* (July 2019).

[51] Jiong Zhu, Ryan A. Rossi, Anup B. Rao, Tung Mai, Nedim Lipka, Nesreen Ahmed, and Danai Koutra. 2021. Graph Neural Networks with Heterophily. In *AAAI*.

[52] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs. *NeurIPS* 33 (2020).